

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

TZ-08 Addendum — Time Skip Architecture (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Дополнение к TZ-08 v0.2 — архитектура промотки времени **Тип:** Техническое задание на миграцию
Цель: реализовать промотку времени через observation policies, НЕ через разные симуляции

0. Главная архитектурная истина

✗ Ошибка (как НЕ делать)

TYPE A = batch execution ← отдельный режим
TYPE B = event skip execution ← отдельный режим
TYPE C = macro simulation ← отдельный симулятор

Это ведёт к: разным правилам причинности, разным результатам мира, hidden divergence.

✓ Правильно

Kernel.execute() — ВСЕГДА одинаковый, deterministic

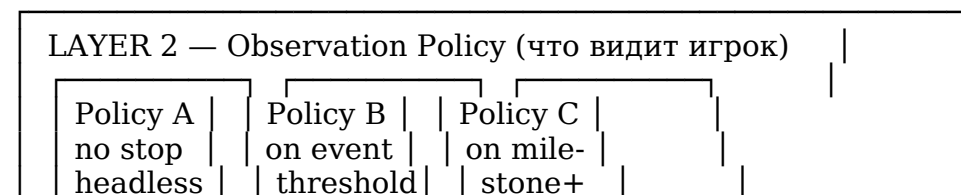
SkipPolicy (observation policy, NOT execution mode):

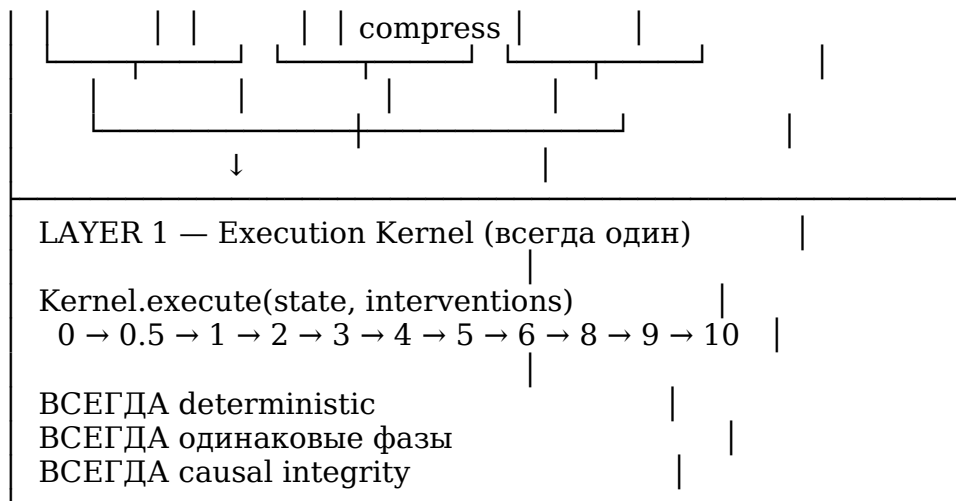
- A → no stop (headless batch)
- B → stop on significance threshold
- C → stop on semantic milestones + aggressive compression

TYPE C — НЕ отдельный симулятор. Это:

kernel.execute()
+ event_filter = SemanticMilestoneFilter
+ state_checkpointing = sparse

1. Двухслойная модель





Принцип

Kernel не знает, что его «проматывают». Kernel просто выполняется. SkipPolicy решает: - когда остановиться - что запомнить (checkpoint density) - что показать игроку (event filter)

2. Три Observation Policies

Policy A — Headless Batch (no stop)

Сценарий: игрок идёт к городу 3 дня.

Что происходит:

$state_t \rightarrow kernel.execute() \times N \rightarrow state_{t+N}$

- Kernel выполняется N раз
- Нет perception (уже вынесен из kernel в TZ-08 v0.2)
- Нет rendering
- Event log: все события записываются, ни одно не показывается
- После skip: игрок видит summary («за 3 дня пути: Люся простудилась, Торнин закрыл таверну»)

Код:

```
class SkipPolicyA:
    """Headless batch. No stops. Full kernel execution."""

    def execute(
        self,
        kernel,
        campaign_id: str,
        scene_state: dict,
```

```

    ticks: int,
    start_tick: int,
) -> TimeSkipResult:
    event_log = []
    _state = scene_state
    _tick = start_tick

    for _ in range(ticks):
        _state = kernel.execute(
            campaign_id=campaign_id,
            scene_state=_state,
            tick_number=_tick,
        )
        # Collect events for summary (no stops)
        _events = _state.get("tick_events", [])
        event_log.extend(_events)
        _tick += 1

    return TimeSkipResult(
        final_state=_state,
        event_log=event_log,
        stop_reason="completed",
        ticks_skipped=ticks,
        stops=[],
    )

```

Гарантии:  Полная causal integrity. Тот же kernel, тот же результат.

Policy B — Event-Threshold Stop

Сценарий: игрок спит 8 часов. Мир идёт вперёд, но останавливается на значимом событии.

Что происходит:

```

for t in range(T):
    state = kernel.execute(headless=True)
    if significance_detector.has_significant_event(state):
        STOP → return state, event, stop_reason

```

- Kernel выполняется
- После каждого тика проверяем: было ли значимое событие?
- Если да — останавливаемся, показываем игроку
- Если нет — продолжаем

Код:

```

class SignificanceDetector:
    """Определяет, когда остановить time skip.

```

НЕ отдельная симуляция. Читает state после kernel.execute().

Significant events:

- NPC death / birth
 - Combat start
 - NPC enters player's location
 - Trauma event (identity_integrity drops below threshold)
 - Faction war declared
 - Marriage / succession
 - NPC speaks about player
- """

```
SIGNIFICANT_EVENT_TYPES = frozenset({
    "npc_death", "npc_birth", "combat_start",
    "faction_war", "marriage", "succession",
    "npc_enters_player_location", "trauma_event",
    "npc_speaks_about_player",
})
```

```
TRAUMA_THRESHOLD = 0.3 # identity_integrity < 0.3 = trauma
```

```
def check(self, state: dict, prev_state: dict) -> Optional[SignificantEvent]:
    """Check if this tick produced a significant event.
```

Pure function: reads state, returns event or None.
Does NOT mutate state.
"""

1. Event-based significance

```
for event in state.get("tick_events", []):
    if event.get("type") in self.SIGNIFICANT_EVENT_TYPES:
        return SignificantEvent(
            type=event["type"],
            tick=state.get("tick_number", 0),
            details=event,
        )
```

2. State-threshold significance (trauma)

```
for npc_id, npc in self._npc_pairs(state, prev_state):
    prev_integrity = prev_npc.get("psyche", {}).get("identity_integrity", 1.0)
    curr_integrity = npc.get("psyche", {}).get("identity_integrity", 1.0)
    if prev_integrity > self.TRAUMA_THRESHOLD >= curr_integrity:
        return SignificantEvent(
            type="trauma_event",
            tick=state.get("tick_number", 0),
            details={"npc_id": npc_id, "prev": prev_integrity, "curr":
curr_integrity},
        )
```

3. Death detection

```

for npc_id, npc in self._npc_pairs(state, prev_state):
    prev_status = prev_npc.get("body_state", {}).get("life_status", "ALIVE")
    curr_status = npc.get("body_state", {}).get("life_status", "ALIVE")
    if prev_status != "DEAD" and curr_status == "DEAD":
        return SignificantEvent(
            type="npc_death",
            tick=state.get("tick_number", 0),
            details={"npc_id": npc_id},
        )

```

```

return None

```

```

def _npc_pairs(self, state, prev_state):
    """Yield (npc_id, curr_npc, prev_npc) pairs."""
    curr_npcs = {n.get("npc_id"): n for n in state.get("all_npcs_raw", [])}
    prev_npcs = {n.get("npc_id"): n for n in prev_state.get("all_npcs_raw", [])}
    for npc_id, curr in curr_npcs.items():
        prev = prev_npcs.get(npc_id, {})
        yield npc_id, curr, prev

```

```

class SkipPolicyB:

```

```

    """Event-threshold stop. Kernel executes, stops on significance."""

```

```

def __init__(self, detector: SignificanceDetector):
    self._detector = detector

```

```

def execute(
    self,
    kernel,
    campaign_id: str,
    scene_state: dict,
    max_ticks: int,
    start_tick: int,
) -> TimeSkipResult:
    event_log = []
    stops = []
    _state = scene_state
    _prev_state = scene_state
    _tick = start_tick

```

```

    for _ in range(max_ticks):
        _state = kernel.execute(
            campaign_id=campaign_id,
            scene_state=_state,
            tick_number=_tick,
        )

```

```

    # Collect events

```

```

    event_log.extend(_state.get("tick_events", []))

```

```

    # Check significance
    significant = self._detector.check(_state, _prev_state)
    if significant:
        stops.append(significant)
        # STOP — return to player
        return TimeSkipResult(
            final_state=_state,
            event_log=event_log,
            stop_reason=significant.type,
            ticks_skipped=_tick - start_tick,
            stops=stops,
            significant_event=significant,
        )

    _prev_state = _state
    _tick += 1

    return TimeSkipResult(
        final_state=_state,
        event_log=event_log,
        stop_reason="max_ticks_reached",
        ticks_skipped=max_ticks,
        stops=stops,
    )

```

Гарантии:  Полная causal integrity. Останавливается на важном.

Policy C — Milestone Sampling + Aggressive Compression

Сценарий: игрок — младенец, проходит 5 лет.

КРИТИЧЕСКОЕ ПРАВИЛО: это НЕ отдельный симулятор. Это:

```

kernel.execute()
+ event_filter = SemanticMilestoneFilter (aggressive)
+ state_checkpointing = sparse (раз в год, не раз в тик)

```

Что происходит:

```

for year in range(years):
    for tick in range(TICKS_PER_YEAR):
        state = kernel.execute(headless=True)

        # Aggressive filter: запоминаем ТОЛЬКО milestones
        milestone = milestone_filter.check(state, prev_state)
        if milestone:
            checkpoints.append((tick, state_copy, milestone))

    prev_state = state

```

```
# End of year: macro summary
year_summary = compress_year(checkpoints)
summaries.append(year_summary)
```

Ключевое отличие от Policy B: - Policy B останавливается на КАЖДОМ значимом событии - Policy C НЕ останавливается, а **запоминает milestones** и продолжает - Игрок видит **summary** после skip, не остановки во время

Код:

```
class SemanticMilestoneFilter:
```

```
    """Aggressive filter for Policy C. Запоминает ТОЛЬКО milestones.
```

```
    НЕ останавливает kernel. Collects checkpoints for later playback.
```

```
    Milestones (для младенчества, но применимо к ЛЮБОМУ периоду):
```

- attachment_formed (привязанность к родителю)*
- trauma_received (травма)*
- separation_event (разлука)*
- discovery (первый раз увидел X)*
- socialization (первый контакт с другими)*
- language_milestone (начал говорить)*
- personality_trait_formed (черта закрепились)*
- loss_event (потеря близкого)*

```
    ВАЖНО: применимо к ЛЮБОМУ long-duration event, не только детству:
```

- long_journey → milestones = encounters, discoveries, hardships*
- imprisonment → milestones = abuse, escape attempts, alliances*
- war → milestones = battles, deaths, betrayals*

```
MILESTONE_TYPES = frozenset({
```

```
    # Childhood
```

```
    "attachment_formed", "trauma_received", "separation_event",
```

```
    "discovery", "socialization", "language_milestone",
```

```
    "personality_trait_formed", "loss_event",
```

```
    # General (any long period)
```

```
    "combat_won", "combat_lost", "ally_gained", "ally_lost",
```

```
    "skill_mastered", "secret_discovered", "betrayal_committed",
```

```
    "reputation_threshold_crossed", "faction_joined", "faction_left",
```

```
})
```

```
    # Personality formation tracking (для identity development)
```

```
DRIVE_CHANGE_THRESHOLD = 0.05 # если drives изменились > 5% =  
milestone
```

```
def check(  
    self,
```

```

state: dict,
prev_state: dict,
context: dict,
) -> Optional[Milestone]:
    """Check if this tick produced a milestone.

Pure function. Does NOT stop kernel. Returns milestone for logging.
    """

    tick = state.get("tick_number", 0)

    # 1. Event-based milestones
    for event in state.get("tick_events", []):
        if event.get("type") in self.MILESTONE_TYPES:
            return Milestone(
                type=event["type"],
                tick=tick,
                details=event,
                requires_playback=self._requires_playback(event["type"]),
            )

    # 2. Drive formation milestones (для младенчества)
    child_id = context.get("child_id")
    if child_id:
        child = self._find_npc(state, child_id)
        prev_child = self._find_npc(prev_state, child_id)
        if child and prev_child:
            drives = child.get("drives", {})
            prev_drives = prev_child.get("drives", {})
            for drive_name in ["control", "significance", "fear", "desire"]:
                curr = drives.get(drive_name, 0.25)
                prev = prev_drives.get(drive_name, 0.25)
                if abs(curr - prev) > self.DRIVE_CHANGE_THRESHOLD:
                    return Milestone(
                        type="personality_trait_formed",
                        tick=tick,
                        details={
                            "npc_id": child_id,
                            "drive": drive_name,
                            "prev": prev,
                            "curr": curr,
                            "delta": curr - prev,
                        },
                        requires_playback=True,
                    )

    # 3. Trauma threshold (для любого возраста)
    for npc_id, curr_npc, prev_npc in self._npc_triples(state, prev_state):
        prev_integrity = prev_npc.get("psyche", {}).get("identity_integrity", 1.0)
        curr_integrity = curr_npc.get("psyche", {}).get("identity_integrity", 1.0)
        if prev_integrity > 0.3 >= curr_integrity:

```



```

        return Milestone(
            type="trauma_received",
            tick=tick,
            details={"npc_id": npc_id, "prev": prev_integrity, "curr":
curr_integrity},
            requires_playback=True,
        )

```

```

    return None

```

```

def _requires_playback(self, milestone_type: str) -> bool:
    """Нужна ли игроку сцена для этого milestone."""
    playback_required = {
        "trauma_received", "separation_event", "loss_event",
        "attachment formed", "personality_trait formed",
        "combat_won", "combat_lost", "ally_lost", "betrayal committed",
    }
    return milestone_type in playback_required

```

```

def _find_npc(self, state, npc_id):
    for npc in state.get("all_npcs_raw", []):
        if npc.get("npc_id") == npc_id or npc.get("id") == npc_id:
            return npc
    return None

```

```

def _npc_triples(self, state, prev_state):
    curr = {n.get("npc_id"): n for n in state.get("all_npcs_raw", [])}
    prev = {n.get("npc_id"): n for n in prev_state.get("all_npcs_raw", [])}
    for nid, c in curr.items():
        yield nid, c, prev.get(nid, {})

```

```

class SkipPolicyC:

```

```

    """Milestone sampling + aggressive compression.

```

```

    НЕ отдельный симулятор. Kernel.execute() + sparse checkpointing.

```

```

    Применимо к ЛЮБОМУ long-duration event:

```

```

    - младенчество (5 лет)
    - долгое путешествие (месяцы)
    - тюремное заключение (годы)
    - война (сезоны)
    """

```

```

def _init_(self, milestone_filter: SemanticMilestoneFilter):
    self._filter = milestone_filter

```

```

def execute(
    self,
    kernel,

```

```

campaign_id: str,
scene_state: dict,
max_ticks: int,
start_tick: int,
context: dict, # {"child_id": "...", "period": "childhood", ...}
) -> TimeSkipResult:
    checkpoints = [] # sparse: только milestones
    event_log = []
    _state = scene_state
    _prev_state = scene_state
    _tick = start_tick

    for _ in range(max_ticks):
        _state = kernel.execute(
            campaign_id=campaign_id,
            scene_state=_state,
            tick_number=_tick,
        )

        # Collect ALL events (for summary compression)
        event_log.extend(_state.get("tick_events", []))

        # Check milestone (aggressive filter — НЕ останавливает)
        milestone = self._filter.check(_state, _prev_state, context)
        if milestone:
            # Sparse checkpoint: сохраняем state ТОЛЬКО на milestones
            import copy
            checkpoints.append(MilestoneCheckpoint(
                tick=_tick,
                state=copy.deepcopy(_state),
                milestone=milestone,
            ))

        _prev_state = _state
        _tick += 1

        # Compress: из N тиков → M milestones (M << N)
        summary = self._compress(checkpoints, event_log, context)

    return TimeSkipResult(
        final_state=_state,
        event_log=event_log,
        stop_reason="completed",
        ticks_skipped=max_ticks,
        stops=[], # Policy C не останавливается
        checkpoints=checkpoints,
        summary=summary,
    )

def _compress(

```

```

self,
checkpoints: list,
event_log: list,
context: dict,
) -> PeriodSummary:
    """Сжимает N тиков в summary для игрока.

    Из 44000 тиков (5 лет) → 5-10 milestones → 1 summary.
    """

    milestones = [cp.milestone for cp in checkpoints]
    playback_scenes = [m for m in milestones if m.requires_playback]

    # Группировка по годам (или периодам)
    periods = self._group_by_period(checkpoints, context)

    return PeriodSummary(
        total_ticks=len(event_log),
        milestone_count=len(milestones),
        playback_scenes=playback_scenes,
        periods=periods,
        highlights=[m.type for m in milestones],
    )

def _group_by_period(self, checkpoints, context):
    """Группирует milestones по периодам (годы, сезоны, etc.)."""
    period_type = context.get("period", "year")
    if period_type == "year":
        # Группировка по игровым годам
        TICKS_PER_YEAR = 24 * 365 # 24 тика/день × 365 дней
        groups = {}
        for cp in checkpoints:
            year = cp.tick // TICKS_PER_YEAR
            groups.setdefault(year, []).append(cp)
        return [
            {"period": f"Year {y}", "milestones": [cp.milestone for cp in cps]}
            for y, cps in sorted(groups.items())
        ]
    return [{"period": "all", "milestones": [cp.milestone for cp in checkpoints]}]

```

Гарантии: ✓ Тот же kernel. ✓ Causal integrity. ⚠ Приблизённая (sparse checkpointing), но не divergent — kernel честно выполняется, просто игрок видит не всё.

3. TimeSkipExecutor — единая точка входа

```

"""
path: backend/app/services/world/time_skip_executor.py
Назначение: Единая точка входа для time skip.
Kernel ВСЕГДА один. SkipPolicy определяет observation.

```

```
"""
```

```
from __future__ import annotations
import logging
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
```

```
logger = logging.getLogger(__name__)
```

```
@dataclass
class SignificantEvent:
    type: str
    tick: int
    details: Dict[str, Any]
```

```
@dataclass
class Milestone:
    type: str
    tick: int
    details: Dict[str, Any]
    requires_playback: bool = False
```

```
@dataclass
class MilestoneCheckpoint:
    tick: int
    state: Dict[str, Any]
    milestone: Milestone
```

```
@dataclass
class PeriodSummary:
    total_ticks: int
    milestone_count: int
    playback_scenes: List[Milestone]
    periods: List[Dict[str, Any]]
    highlights: List[str]
```

```
@dataclass
class TimeSkipResult:
    final_state: Dict[str, Any]
    event_log: List[Dict[str, Any]]
    stop_reason: str
    ticks_skipped: int
    stops: List[SignificantEvent]
    significant_event: Optional[SignificantEvent] = None
```

checkpoints: Optional[List[MilestoneCheckpoint]] = None
summary: Optional[PeriodSummary] = None

class TimeSkipExecutor:

"""Единая точка входа для time skip.

Kernel.execute() — ВСЕГДА одинаковый.

SkipPolicy — определяет observation (когда остановиться, что запомнить).
"""

def __init__(self, kernel):

self._kernel = kernel

self._policy_a = SkipPolicyA()

self._policy_b = SkipPolicyB(SignificanceDetector())

self._policy_c = SkipPolicyC(SemanticMilestoneFilter())

def skip(
self,

campaign_id: str,

scene_state: dict,

ticks: int,

start_tick: int,

policy: str = "A", # "A" / "B" / "C"

context: Optional[dict] = None,

) -> TimeSkipResult:

"""Промотать время. Kernel выполняется, policy определяет observation.

Args:

policy: "A" (headless batch), "B" (stop on event), "C" (milestone sampling)

context: доп. контекст для Policy C (child_id, period, etc.)

"""

logger.info(
f"[TIME_SKIP] policy={policy} ticks={ticks} start={start_tick} "

f"context={context}"

)

if policy == "A":

return self._policy_a.execute(
self._kernel, campaign_id, scene_state, ticks, start_tick,

)

elif policy == "B":

return self._policy_b.execute(
self._kernel, campaign_id, scene_state, ticks, start_tick,

)

)

elif policy == "C":

return self._policy_c.execute(
self._kernel, campaign_id, scene_state, ticks, start_tick,

context or {},

)

)

```

else:
    raise ValueError(f"Unknown skip policy: {policy}")

```

4. Когда какую Policy применять

Сценарий	Policy	Почему
Путешествие 3 дня	A	Нет ожидаемых событий, нужен просто результат
Сон 8 часов	B	Мир может произвести значимое событие (NPC подходит, война начинается)
Ожидание NPC	B	NPC может вернуться, умереть, предать
Младенчество 5 лет	C	Много тиков, мало «событий», но много формирования. Milestones + compression
Долгое путешествие (месяцы)	C	Много тиков, milestones = encounters, discoveries
Тюремное заключение	C	Много тиков, milestones = abuse, escape, alliances
Война (сезоны)	C	Много тиков, milestones = battles, deaths, betrayals
Быстрый пропуск (1-2 часа)	A	Короткий период, нет смысла в milestones
СК succession (годы)	C	Много тиков, milestones = succession, faction changes

Правило выбора:

```

if ticks < 100:
    policy = "A" # короткий — batch
elif expect_significant_events:
    policy = "B" # средний с событиями — stop on threshold

```

else:

policy = "C" # длинный без явных событий — milestone sampling

5. Event Log — что видит игрок после skip

После любого skip игрок видит **TimeSkipResult**:

Policy A

"За 3 дня пути:

- Люся простудилась (день 2)
- Торнин закрыл таверну на ремонт (день 3)
- Ты прибыл в городские ворота"

Policy B (остановилась на событии)

"Ты спал 4 часа, когда:

- Тень постучал в твою дверь (час 4)

[Остановлено: NPC enters player location]"

Policy C (milestones + summary)

"За 5 лет младенчества:

Год 1:

- Привязанность к матери сформирована
- Первая улыбка

Год 2:

- Травма: отец ударил мать при тебе
- Начал бояться громких голосов

Год 3:

- Первые слова
- Социализация: встретил других детей

Год 4:

- Травма: мать ушла (не вернулась)
- Замкнулся

Год 5:

- Черта характера закрепились: недоверие
- Готов к обучению

[5 milestones требуют отыгрыша — показать сцены?]"

6. СВОДКА ВНЕДРЕНИЯ

Компонент	Файл	Время
TimeSkipResult + dataclasses	time_skip_executor.py	30 мин
SkipPolicyA (headless batch)	time_skip_executor.py	30 мин
SignificanceDetector	time_skip_executor.py	1 час
SkipPolicyB (event-threshold)	time_skip_executor.py	30 мин
SemanticMilestoneFilter	time_skip_executor.py	1.5 часа
SkipPolicyC (milestone + compress)	time_skip_executor.py	1.5 часа
TimeSkipExecutor (entry point)	time_skip_executor.py	30 мин
Tests	test_time_skip.py	1 час

Итого: ~7 часов.

7. ПРИНЦИПЫ

1. **Kernel ВСЕГДА один** — execute() не знает про skip
 2. **SkipPolicy = observation policy** — когда остановиться, что запомнить
 3. **Policy C — НЕ отдельный симулятор** — kernel + filter + sparse checkpointing
 4. **Применимо к ЛЮБОМУ long-duration event** — не только младенчество
 5. **Causal integrity сохраняется** — kernel честно выполняется во всех policies
 6. **Игрок видит разное** — A: summary, B: stop + event, C: milestones + summary
-

8. ФИНАЛЬНАЯ ПРОВЕРКА

1. TimeSkipExecutor exists

```
python -c "
```

```
import sys; sys.path.insert(0, 'backend')
```

```
from app.services.world.time_skip_executor import TimeSkipExecutor
```

```
print('TimeSkipExecutor OK')
```

```
"
```

2. All 3 policies exist


```
python -c "
import sys; sys.path.insert(0, 'backend')
from app.services.world.time_skip_executor import SkipPolicyA, SkipPolicyB,
SkipPolicyC
print('All policies OK')
"
```

3. SemanticMilestoneFilter exists

```
python -c "
import sys; sys.path.insert(0, 'backend')
from app.services.world.time_skip_executor import SemanticMilestoneFilter
f = SemanticMilestoneFilter()
assert 'attachment_formed' in f.MILESTONE_TYPES
assert 'trauma_received' in f.MILESTONE_TYPES
print('MilestoneFilter OK')
"
```

4. SignificanceDetector exists

```
python -c "
import sys; sys.path.insert(0, 'backend')
from app.services.world.time_skip_executor import SignificanceDetector
d = SignificanceDetector()
assert 'npc_death' in d.SIGNIFICANT_EVENT_TYPES
print('SignificanceDetector OK')
"
```

9. ОБНОВЛЁННЫЙ ПОРЯДОК ПРИМЕНЕНИЯ ВСЕХ ДОКУМЕНТОВ

Полный список инструкций (12 документов)

ФАЗА 1 — Фундамент (~27 часов)

└─ TZ-01 (срочные фиксы)	~5 ч	✓ применён
└─ TZ-08 v0.2 (WorldTickKernel strict)	~8 ч	← СЛЕДУЮЩИЙ
└─ TZ-08 Addendum (Time Skip Architecture)	~7 ч	← после TZ-08 v0.2
└─ TZ-03 (frontend↔backend контракт)	~8 ч	

ФАЗА 2 — Deterministic kernel (~6 часов)

└─ Kernel Isolation Repair	~6 ч
----------------------------	------

ФАЗА 3 — Causal chains (~25 часов)

└─ TZ-02 (каузальная труба воли)	~12 ч
└─ TZ-04 (spatial authority)	~6 ч
└─ TZ-05 (LLM/DM contract)	~7 ч

ФАЗА 4 — Cleanup (~10 часов)

└─ TZ-06 (dead code cleanup)	~7 ч
└─ TZ-07 (documentation drift)	~3 ч

ФАЗА 5 — Dynastic simulation (~10 часов)

ИТОГО: ~78 часов = ~10 рабочих дней

Порядок исполнения файлов

ШАГ 1. TZ-01 — Критические срочные фиксы

Файл: TZ-01_Final_Patches.pdf

Статус: ☒ ПРИМЕНЁН

Что: NameError, ImportError, хардкоды, fail-fast

Время: 5 ч

ШАГ 2. TZ-08 v0.2 — WorldTickKernel Unification

Файл: TZ-08_v0.2_Strict_Kernel_Instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН (СЛЕДУЮЩИЙ)

Что: InterventionEvent, _run_core_phases, RulesSubscriber, perception outside kernel

Время: 8 ч

Зависимости: TZ-01 (применён ☒)

ШАГ 3. TZ-08 Addendum — Time Skip Architecture

Файл: TZ-08_Addendum_Time_Skip_Architecture.pdf (этот документ)

Статус: ☐ НЕ ПРИМЕНЁН

Что: TimeSkipExecutor, SkipPolicyA/B/C, SignificanceDetector, SemanticMilestoneFilter

Время: 7 ч

Зависимости: TZ-08 v0.2 (нужен unified kernel)

ШАГ 4. TZ-03 — Frontend↔Backend Contract Repair

Файл: TZ-03_Contract_Repair_Instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: GameActionResponse, npc_positions Dict, cue_key, frontend authority removal, Spatial Oracle, SSE

Время: 8 ч

Зависимости: TZ-01 (применён ☒), TZ-08 v0.2 (нужен unified execute)

ШАГ 5. Kernel Isolation Repair

Файл: Kernel_Isolation_Repair_Instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: KernelRNG, _TickContext.rng, DecisionHub(rng=...), random.* → ctx.rng.*, SUPERBOX validation

Время: 6 ч

Зависимости: TZ-08 v0.2 (нужен unified _run_core_phases)

ШАГ 6. TZ-02 — Каузальная труба воли и психика NPC

Файл: TZ-02_Final_Patches.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: BUG-001 (directive fields), apply_drives_mutation → L1, ResolutionEngine effective_drives, BreakProgressEngine, BehaviorMask, L1Chronicle persistence, HP unification

Время: 12 ч

Зависимости: TZ-08 v0.2 (нужен Phase 2 в player turn),
Kernel Isolation (нужен deterministic RNG)

ШАГ 7. TZ-04 — Spatial Authority & Physics Repair

Файл: tz-04-spatial-authority-instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: zombie readers → SpatialQueryService, RNG → KernelRNG,
dead modules removal, SpatialFactory, SceneChange mutations

Время: 6 ч

Зависимости: TZ-08 v0.2 (нужен unified pipeline),
Kernel Isolation (нужен KernelRNG)

ШАГ 8. TZ-05 — LLM Contract & DM-Agent Repair

Файл: tz-05-llm-contract-instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: dm_agent import, scene_outcome_builder NameError,
JSON recovery, recent_text, forbidden-список в промпт,
MockProvider exclusion, dead code removal

Время: 7 ч

Зависимости: TZ-01 (применён ☒)

ШАГ 9. TZ-06 — Dead Code Cleanup & Engineering Debt

Файл: tz-06-cleanup-instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: 5 мёртвых модулей, 13 мёртвых методов, print→logger,
magic numbers → constants, i18n, WillState unification,
sprite_resolver merge, RelationshipStore LRU

Время: 7 ч

Зависимости: TZ-04, TZ-05 (нужно сначала удалить то, что
они пометили как мёртвое)

ШАГ 10. TZ-07 — Documentation Drift & ADR Revision

Файл: tz-07-documentation-instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: удалить 12.md, архивировать CAUSAL_CONTRACT v1.0 +
ТЕХЗАДАНИЕ ПРЕЕМНИКУ, ADR_STATUS_MATRIX,
обновить README, KNOWN_ISSUES статусы

Время: 3 ч

Зависимости: TZ-06 (нужно сначала почистить код,
потом документировать результат)

ШАГ 11. CK CORE LAYER SPEC — Dynastic Simulation Engine

Файл: CK_CORE_LAYER_SPEC_Instructions.pdf

Статус: ☐ НЕ ПРИМЕНЁН

Что: WorldChronicle, Lineage, AgingEngine, BirthGenerator,
SuccessionPipeline, CrossNPCRelationships,
NPCState CK fields, tick_orchestrator integration

Время: 10 ч

Зависимости: ВСЕ предыдущие (нужен deterministic kernel,
unified pipeline, spatial authority, LLM contract)

ШАГ 12. (будущее) СК GAMEPLAY LAYER

Файл: не создан

Что: player lineage UI, marriage system, faction dynamics,
settlement management, death screen → heir selection,
calendar UI, family tree

Время: TBD

Зависимости: СК CORE LAYER

Визуальная карта зависимостей

TZ-01 ✓



TZ-08 v0.2 ← СЛЕДУЮЩИЙ



TZ-08 Addendum (Time Skip)



TZ-03 (frontend↔backend) —————



Kernel Isolation



TZ-02 (воле) ——— TZ-04 (spatial) ———



—— TZ-05 (LLM) ———



TZ-06 (cleanup) ←—————



TZ-07 (docs)



СК CORE LAYER



СК GAMEPLAY LAYER (будущее)

КОНЕЦ TZ-08 ADDENDUM